Microsoft Agent Framework & Foundry Agent Service

A comprehensive guide to building, orchestrating, and deploying enterprise-grade AI agents with Microsoft's unified agentic AI platform.

November 2026

Technical Overview & Guide

Table of Contents

1. Executive Summary

2. What is Microsoft Agent Framework (MAF)?

3. MAF Architecture & Core Concepts

4. What is Microsoft Foundry Agent Service?

5. Foundry Agent Service — Key Capabilities

6. How MAF & Foundry Agent Service Work Together

7. Comparison with Other Frameworks

8. Getting Started

9. Use Cases & Real-World Applications

10. Summary & Key Takeaways

1. Executive Summary

Microsoft's agentic AI strategy centers on two complementary pillars: the **Microsoft Agent Framework (MAF)** for designing and coding AI agents, and the **Microsoft Foundry Agent Service** for running, scaling, and governing them in production. Together, they form a complete lifecycle platform for enterprise AI agents.

MAF was publicly previewed in October 2025 and reached version 1.0 General Availability on April 3, 2026. It unifies the previously separate AutoGen and Semantic Kernel frameworks into a single, MIT-licensed open-source SDK available for Python, .NET (C#), and Go.

Foundry Agent Service became generally available in May 2025, providing a fully managed runtime built on Azure that handles hosting, scaling, identity, observability, and enterprise security — so developers can focus entirely on agent logic.

MAF 1.0 GA	Languages	License	Standards
April 3, 2026	Python, C#, Go	MIT (Open Source)	MCP, A2A, AG-UI

2. What is Microsoft Agent Framework (MAF)?

Microsoft Agent Framework is an **open-source SDK and runtime** for building, orchestrating, and deploying AI agents and multi-agent workflows. It is the official successor to both AutoGen (moved to maintenance mode October 2025) and Semantic Kernel.

Why Microsoft Built MAF

For years, .NET and Python developers faced a confusing choice between two Microsoft frameworks with overlapping goals:

- **Semantic Kernel** — Enterprise-grade AI orchestration with a plugin model and deep Azure integration.
- **AutoGen** — Research-oriented multi-agent framework with flexible conversation patterns and group chat.

MAF merges AutoGen's agent abstractions and group-chat patterns with Semantic Kernel's enterprise observability and plugin model into **one SDK, one managed runtime**.

The Competitive Landscape

Microsoft's MAF arrived alongside similar moves from other major players:

Framework	Released	Description
OpenAI Agents SDK	March 2025	Python-first agent toolkit from OpenAI
Google ADK	April 2025	Agent Development Kit for Vertex AI
Microsoft MAF	October 2025	Unified AutoGen + Semantic Kernel
LangGraph v1.2	May 2026	Stateful graph-based agent runtime

3. MAF Architecture & Core Concepts

Layered Architecture

MAF is structured as a clean stack of layers, each with a distinct responsibility:

Layer	Components & Purpose
Your Application	AIAgent, Tools, Sessions, Workflows — the agent logic you write
Microsoft.Agents.AI	The unified agent runtime that orchestrates execution
Microsoft.Extensions.AI	Abstractions layer: IChatClient, AIFunction interfaces
Providers	Azure OpenAI, OpenAI, Anthropic, AWS Bedrock, Ollama, etc.

Core Concepts

- **AIAgent (IAIAgent)** — The core abstraction representing a single agent. An agent wraps a chat client, tools, and instructions into one composable unit.
- **Tools (AIFunction)** — Functions the agent can invoke to interact with external systems, databases, APIs, or run computations.
- **Sessions** — Manage conversation history and state across turns. InMemoryChatHistoryProvider for development; persistent stores for production.
- **Workflows** — Orchestrate multi-step, multi-agent pipelines with branching, looping, and human-in-the-loop patterns.
- **Middleware** — Intercept and transform agent behavior for logging, guardrails, rate limiting, and policy enforcement.
- **Context Providers** — Inject dynamic context (user data, time, location) into agent prompts at runtime without hardcoding.

Supported Model Providers

Provider	Notes
Microsoft Foundry / Azure OpenAI	Native, first-class support
OpenAI (Direct)	GPT-4o, GPT-4, o-series models
Anthropic	Claude models via API
AWS Bedrock	Multiple model providers
Ollama	Local/self-hosted open-source models

Open Standards Support

- **MCP (Model Context Protocol)** — Standardized way for agents to connect to external tools and data sources. Default at MAF 1.0.
- **A2A (Agent-to-Agent)** — Protocol for agents to discover, communicate with, and delegate tasks to other agents across systems.
- **AG-UI (Agent-User Interface)** — Open protocol standardizing how agents communicate with user interfaces for interactive, stateful experiences.
- **OpenAPI** — Import existing REST API definitions as agent tools automatically.

4. What is Microsoft Foundry Agent Service?

Microsoft Foundry Agent Service is a **fully managed cloud service** within Microsoft Foundry (formerly Azure AI Foundry) for deploying, running, scaling, and governing AI agents in enterprise production environments.

If MAF is where you **write** the agent, Foundry Agent Service is where you **run** it. Microsoft handles all infrastructure concerns — compute, storage, scaling, identity, and security — so developers focus purely on agent logic and business value.

Timeline

Date	Milestone	Details
May 2025	General Availability (v1)	Initial GA of Agent Service in Azure AI Foundry
October 2025	MAF Integration	Aligned with MAF public preview for unified development
2026	Next-Gen GA	Redesigned API format, hosted agents, enhanced runtime

What Problems Does It Solve?

Moving AI agents from prototype to production is the #1 challenge in enterprise AI. Foundry Agent Service addresses this by providing:

- **Infrastructure Management** — No need to provision or manage VMs, containers, or databases
- **Conversation State** — Built-in thread storage maintains context across interactions
- **Enterprise Security** — Microsoft Entra ID, RBAC, audit logs, conditional access
- **Scaling** — Automatic horizontal scaling based on demand
- **Observability** — Built-in tracing, evaluation, and monitoring (AgentOps)
- **Multi-Channel Deployment** — Deploy to Teams, Microsoft 365 Copilot, custom apps, or APIs

5. Foundry Agent Service — Key Capabilities

Capability	Description
Hosted Agents	Containerized agentic applications running in secure sandboxes with full lifecycle management
Tool Calling	Automatic tool invocation, code interpreter, file retrieval, and custom function integration
Thread Storage	Bring Your Own (BYO) thread storage with Azure Cosmos DB for conversation persistence
AgentOps	Tracing, evaluation, and monitoring to optimize agent behavior in production
Identity & Governance	Microsoft Entra with full RBAC, audit logs, and enterprise conditional access
Agent 365 Integration	Control plane for AI agents integrated into Microsoft 365 Admin Center
MCP Server Support	Connect agents to enterprise tools and data sources via Model Context Protocol
Multi-Model	Works with Azure OpenAI, OpenAI, and other AI models via Foundry model catalog

Foundry Agent Service Resources

When you create an agent through Foundry Agent Service, the platform automatically provisions the necessary cloud resources:

- **Compute** — Managed containers for agent execution (no VM setup required)
- **Storage** — Thread storage, knowledge bases, and file management
- **Networking** — Secure endpoints with VNet integration options
- **Monitoring** — Application Insights integration for real-time observability
- **Security** — Managed identity, key vault integration, and data encryption at rest and in transit

6. How MAF & Foundry Agent Service Work Together

A common point of confusion is whether MAF and Foundry Agent Service are alternatives. They are not — they are **complementary layers** that together form a complete agent platform:

Dimension	Microsoft Agent Framework	Foundry Agent Service
Role	Design & implement agent logic	Operate, scale & govern agents
Type	Open-source SDK (code library)	Managed cloud service (Azure)
Scope	Agent code, tools, workflows	Runtime, security, monitoring, channels
Analogy	The application you write	The server that runs it
License	MIT (free, open-source)	Azure consumption-based pricing
Language	Python, C#, Go	Language-agnostic (runs containers)

The Development-to-Production Flow

1. **Step 1: Design** — Use MAF to define agents, their tools, instructions, and orchestration patterns.
2. **Step 2: Develop** — Write agent code in Python or C# using MAF's SDK. Test locally with in-memory providers.
3. **Step 3: Package** — Containerize your agent application as a Docker image.
4. **Step 4: Deploy** — Push to Foundry Agent Service. It provisions compute, storage, and networking.
5. **Step 5: Connect** — Wire up channels (Teams, Copilot, custom APIs) and data sources via MCP.
6. **Step 6: Monitor** — Use AgentOps for tracing, evaluation, and continuous improvement.
7. **Step 7: Govern** — Manage access, compliance, and policies through Entra ID and Agent 365.

7. Comparison with Other Frameworks

Understanding how MAF compares to other popular agent frameworks helps teams make informed choices:

Dimension	MAF	LangGraph	CrewAI	LangChain
Languages	Python, C#, Go	Python, TypeScript	Python	Python
License	MIT	MIT	Apache 2.0	MIT
Best For	Azure/Microsoft stack, enterprise governance	Stateful workflows, compliance-heavy agents	Role-based agent teams, rapid prototyping	RAG pipelines, integrations
Ramp Time	1–2 weeks	2–4 weeks	2–4 days	~1 week
Token Efficiency	Moderate	Highest	Lower (3–5x overhead)	High
MCP Support	Yes (native)	Yes	Yes	Yes
A2A Support	Yes (native)	Community	Community	Community
Managed Runtime	Foundry Agent Service	LangGraph Cloud	CrewAI Enterprise	LangSmith
Multi-Agent	Built-in workflows	Explicit graph wiring	Implicit role delegation	Via LangGraph

When to Choose MAF

- ✓ Your organization is already on the Microsoft / Azure stack
- ✓ You need .NET (C#) support — most other frameworks are Python-only
- ✓ Enterprise governance (Entra ID, RBAC, Agent 365) is a hard requirement
- ✓ You are migrating from AutoGen or Semantic Kernel
- ✓ You want a single vendor path from code to managed production runtime

When to Consider Alternatives

- → **LangGraph** — Maximum control over graph topology; best token efficiency; Python/TypeScript focus
- → **CrewAI** — Fastest time-to-prototype for role-based agent teams; simplest learning curve
- → **LangChain** — Broadest integration library (1,000+ connectors); ideal as a substrate layer

8. Getting Started

Quick Start with MAF (Python)

Install the MAF Python package and create your first agent in just a few lines:

```
# Install
pip install agent-framework

# Create an agent
from agent_framework import Agent
from agent_framework.openai import OpenAIChatClient

agent = Agent(
    name="assistant",
    instructions="You are a helpful assistant.",
    chat_client=OpenAIChatClient(model="gpt-4o")
)

# Run
session = await agent.CreateSessionAsync()
response = await agent.RunAsync("Hello!", session)
```

Quick Start with MAF (.NET / C#)

```
# Install
dotnet add package Microsoft.Agents.AI

// C# Agent Setup
var agent = new AIAgentBuilder()
    .WithName("assistant")
    .WithInstructions("You are a helpful assistant.")
    .WithChatClient(chatClient)
    .Build();
```

Deploying to Foundry Agent Service

- **Create a Foundry Project** in your Azure subscription via the Microsoft Foundry portal
- **Configure your agent** with model selection, tools, and knowledge sources
- **Deploy** using the Foundry SDK, CLI, or the portal's visual interface
- **Connect channels** — Teams, Copilot, custom web apps, or REST APIs
- **Monitor** with built-in AgentOps dashboards for tracing and evaluation

Key Resources

→ **GitHub** — github.com/microsoft/agent-framework

→ **Documentation** — learn.microsoft.com/en-us/agent-framework/overview

→ **Foundry Docs** — learn.microsoft.com/azure/ai-foundry/agents

→ **PyPI Package** — agent-framework

→ **NuGet Package** — Microsoft.Agents.AI

9. Use Cases & Real-World Applications

Customer Service Agents

Build intelligent agents that handle customer queries across channels (Teams, web chat, email) with access to knowledge bases, CRM systems, and order management via MCP tools. Agents maintain conversation context and escalate to humans when needed.

Internal IT Help Desk

Deploy agents that troubleshoot common IT issues, reset passwords, provision software, and manage tickets. Integration with ServiceNow, Jira, and Azure AD via tool calling. Governed through Entra ID and Agent 365.

Data Analysis & Reporting

Create agents with code interpreter tools that analyze datasets, generate visualizations, and produce reports. Multi-agent workflows can coordinate a data-gathering agent, an analysis agent, and a report-writing agent.

Sales & CRM Automation

Agents that prepare meeting briefs, update CRM records, draft follow-up emails, and surface pipeline insights. Connected to Salesforce, HubSpot, or Dynamics via MCP servers.

Document Processing Pipelines

Multi-agent systems that extract data from invoices, contracts, and forms, then validate, classify, and route them through approval workflows with human-in-the-loop checkpoints.

Software Development Assistants

Coding agents that review PRs, suggest fixes, generate tests, and update documentation. Integrated with GitHub, Azure DevOps, and CI/CD pipelines through tool functions.

10. Summary & Key Takeaways

Microsoft's agentic AI platform is built on a clear separation of concerns:

Microsoft Agent Framework

The **code layer** — the open-source SDK you use to design, build, and test AI agents. Unifies AutoGen + Semantic Kernel. Supports Python, C#, and Go. MIT licensed.

Foundry Agent Service

The **platform layer** — the fully managed Azure service that runs, scales, secures, and monitors your agents in production. Handles infrastructure so you focus on logic.

Key Takeaways

- MAF and Foundry Agent Service are **complementary, not competing** — one is the SDK, the other is the runtime.
- MAF is the **official successor** to both AutoGen and Semantic Kernel — consolidating Microsoft's agent story.
- The framework supports **open standards** (MCP, A2A, AG-UI) ensuring interoperability beyond the Microsoft ecosystem.
- Foundry Agent Service provides **enterprise-grade governance** through Entra ID, RBAC, Agent 365, and built-in compliance.
- **.NET/C# support** is a key differentiator — most competing frameworks are Python-only.
- The platform is **model-agnostic** — it works with Azure OpenAI, OpenAI, Anthropic, Bedrock, and local models via Ollama.